Solution: Inorder Successor in BST

Problem Statement

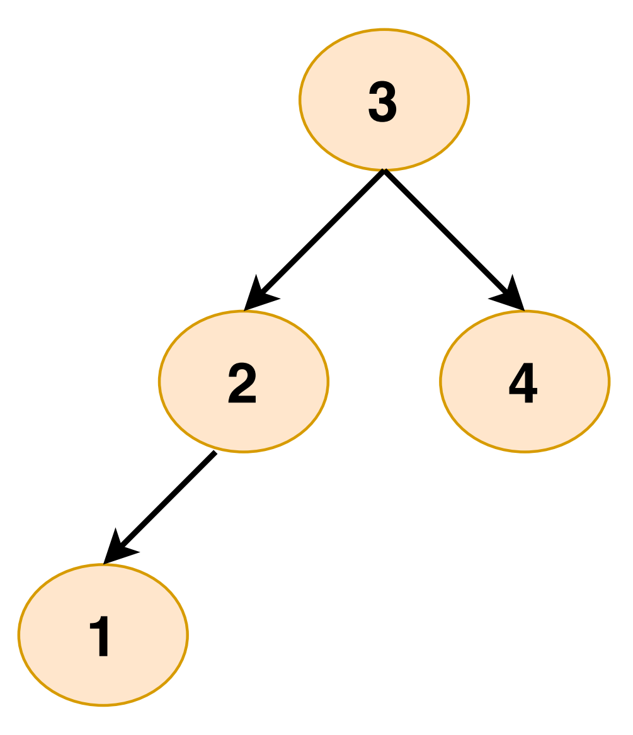
Given a `root` node of the binary search tree and node `p`, return the value of the in-order successor of node `p` in the given tree. If the given node has no in-order successor, return `-1`.

The `in-order` successor of a node is the next node in the in-order traversal of the BST, which means it is the node with the `smallest` value `greater` than the given node.

Examples

Example 1:

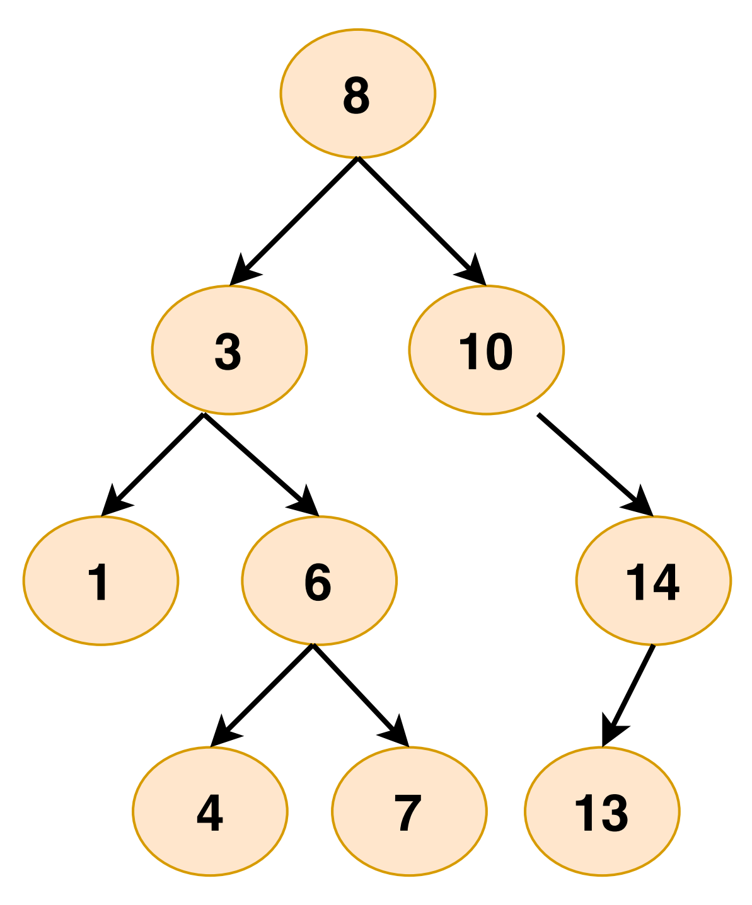
- **Input**: root = [3,2,4,1], p = 2



- **Expected Output**: **3**
- **Justification**: In the in-order traversal [1,2,3,4], the next node after 2 is **3**, making it the in-order successor.

Example 2:

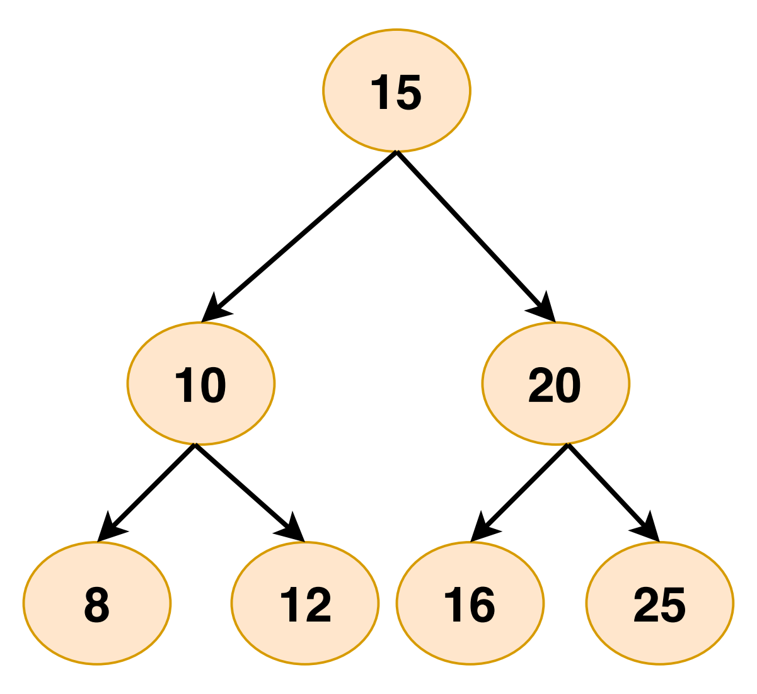
- **Input**: root = [8,3,10,1,6,null,14,null,null,4,7,13], p = 6



- **Expected Output**: **10**
- **Justification**: The in-order traversal of the tree is [1,3,4,6,7,8,10,13,14]. The smallest value greater than 6 is **7**, making it the in-order successor.

Example 3:

- **Input**: root = [15,10,20,8,12,16,25], p = 25



- **Expected Output**: **-1**
- **Justification**: In the in-order traversal [8,10,12,15,16,20,25], there is no node after 25. Hence, the expected output is **-1**.

Solution

To address the problem of finding the in-order successor in a Binary Search Tree (BST), our strategy capitalizes on the BST's ordered nature. The in-order successor of a node is the node that appears immediately after the given node when the tree is traversed in an in-order manner (left, root, right).

For any node p, if it has a right subtree, its in-order successor is the left-most node in its right subtree. Conversely, if the node has no right subtree, the successor is one of its ancestors, specifically the one that p would be the left child of in the path from root to p. This insight leads to a two-fold approach: first, descend towards the given node p, tracking the last node encountered that is greater than p as a potential successor; second, if p has a right child, override the tracked ancestor with the left-most child of p's right subtree. This method is efficient because it traverses each relevant part of the tree at most once, directly homing in on the successor by leveraging the BST property that left children are less than their parents and right children are greater.

Step-by-Step Algorithm

- Start from the root of the BST.
- Initialize a variable to store the potential successor as `-1` or an equivalent value indicating no successor found.
- While the current node is not null:
  - If the value of the current node is less than or equal to the p, move to the right child.
  - If the value of the current node is greater than the p, update the potential successor to the current node and move to the left child.
- Continue this process until the entire tree is traversed or the exact match is found.
- Return the value of the potential successor if found; otherwise, return -1 indicating no successor.

Algorithm Walkthrough

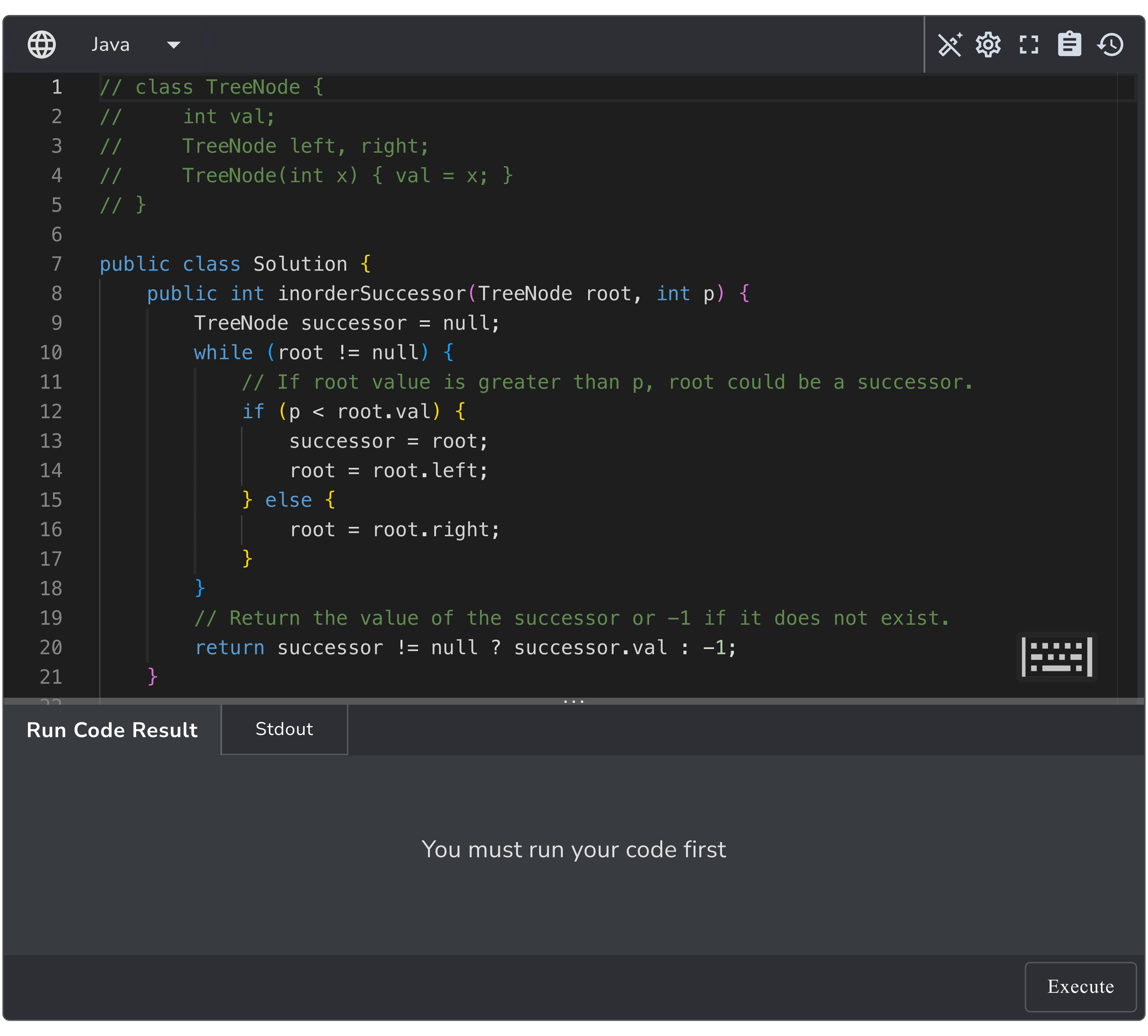
Let's consider the input: root = [8,3,10,1,6,null,14,null,null,4,7,13], p = 6

Let's walk through the algorithm to find the in-order successor of `p = 6` using the same tree structure as before:

```
      8
    /   \
   3     10
  / \      \
 1   6      14
    / \    /
   4   7  13
```

1. **Start at the root** of the BST, which is `8`. Since `6` is less than `8`, we move to the left child of `8`, which is `3`, to find `6`. `successor` = 8
2. **Move to the right child** of `3`, as `3` is not greater than `6`.
3. **Move to the right child** of `6`, as `6` is not greater than `6`.
4. **Inspect the right subtree of** `6`, which leads us to `7`. Since `7` does not have a left child, it is the left-most node in the right subtree of `6` and thus the in-order successor of `6`.
5. **End of Algorithm:** The in-order successor of the node with value `6` in the given BST is `7`.

Code



```java
// class TreeNode {
//     int val;
//     TreeNode left, right;
//     TreeNode(int x) { val = x; }
// }

public class Solution {
    public int inorderSuccessor(TreeNode root, int p) {
        TreeNode successor = null;
        while (root != null) {
            // If root value is greater than p, root could be a successor.
            if (p < root.val) {
                successor = root;
                root = root.left;
            } else {
                root = root.right;
            }
        }
        // Return the value of the successor or -1 if it does not exist.
        return successor != null ? successor.val : -1;
    }
```

You must run your code first

Complexity Analysis

- **Time Complexity**: The algorithm has a time complexity of $O(h)$, where h is the height of the tree. This is because we are traversing the tree from root to leaf, and in the worst case, we might have to traverse the height of the tree.
- **Space Complexity**: The space complexity is $O(1)$ since we are only using a constant amount of extra space for the variables, regardless of the size of the input tree.

← Previous
Inorder Successor In BST (Medium)

Next →
Distribute Coins In Binary Tree (Medium)

☐ Mark as Completed